survey of parallel programming infrastructures

survey of parallel programming infrastructures

why there are so many parallel languages

- in general, **sequential programmining** is more easy because we can traslate the sequential algorithm from pseudo-code to a high-level language like C++
  - this is possible because all sequential machines, at their core, can be abstracted as a **von Neumann Architecture**
    - this single, unified model is the target for all our sequential code
  - even if the real processors differ a lot, the compiler can manage this gap
    - we write our C++ code once, and we trust the compiler (like GCC or Clang) to be smart enough to optimize that same source code for any given architecture
    - the compiler 'fills the gap' between the abstract von Neumann model and the physical, optimized hardware
- when we try to apply this same thinking to parallel programming, this doesn't work
  - why?
    - although parallel architectures are composed of equivalent von Neumann Machines
    - real architectures differ so much that compilers can no longer fill the gap between an abstract parallel model and the real hardware implementation
      - optimized applications can not be generated starting from generic parallel code
      - code extensions have been specialized for particular types of applications/architectures
  - solution
    - this disparity has led to specialization in parallel programming
    - two different approaches
      - **start from scratch (new languages)**
        - because parallel programming is a completely different paradigm, we must create a new languages to handle it
        - this approach did not have big success
          - writing an optimizing compiler is incredibly difficult because these new languages had compilers that were simply 'too immature'
          - there is the adoption problem: we have to convince milions of C++ developers to "learn a new language"
      - **build on what works (extensions)**
        - instead of creating a new language, we create extensions to existing, successful languages (as OpenMP or CUDA)
        - this was a huge success for two reasons
          - developers can easily adopt it. It's just a few new keywords (like pragma) in the C++ code they already know
          - it can be easily integrated into the existing, mature compilers we already trust (like GCC)
    - this is a compromise
      - these extensions are very good at describing some types of parallelism but they make other types very difficult to describe

pros and cons of different technologies

- **Verilog/VHDL**
  - **pros**
    - complete control on computation and memory
    - no overhead introduced in the computation
    - provides access to potentially large computational power
  - **cons**
    - requires specific Hardware (e.g., ASIC or FPGA) to implement functionality
    - difficult to learn: completely different programming language and programming paradigm
    - depends on the chosen target architecture
- **MPI**
  - **pros**
    - can be adopted on different types of architectures
    - scalable solutions
    - synchronization and data communication are explicitly managed
  - **cons**
    - communication can introduce significant overhead
    - programming paradigm more difficult than shared memory based ones
    - standard does not reflect immediately advances in architecture characteristics
- **PThread**
  - **pros**
    - can be adopted on different architectures
    - explicit parallelism and full control over application
  - **cons**
    - task management overhead can be significant
    - not easily scalable solutions
    - low level API
- **OpenMP**
  - **pros**
    - easy to learn
    - scalable solution
    - parallel applications can also be executed sequentially
  - **cons**
    - mainly focused on shared memory homogeneous systems
    - require small interaction between tasks
- **CUDA**
  - **pros**
    - provides access to the computational power of GPUs
    - writing a CUDA kernel is quite easy
    - already optimized libraries
  - **cons**
    - targets only NVIDIA GPUs
    - difficult to extract massive parallelism from application
    - difficult to optimize CUDA kernel
- **OpenCL**
  - **pros**
    - target-independent standard
    - hides architecture details
    - same programming infrastructure for very heterogeneous architecture: CPU + GPU (+FPGA)
  - **cons**
    - difficult programming paradigm for its heterogeneity
    - hiding of architecture details makes difficult to obtain best performances
- **Apache Spark**
  - **pros**
    - API for different languages
    - explicit parallelization and communication are not required
  - **cons**
    - suitable only for big data applications
    - does not (yet) fully support GPUs

Tecnologia	Anno	Parallelismo (Bit, Instr., Task)	Architettura Target Principale	Comunicazione (Memoria, Task)	Vantaggi Chiave (Pro)
Verilog/VHDL	1984/1987	Sì, Sì, No	ASIC – FPGA	Esplicita, Esplicita	Controllo completo; nessuna overhead; grande potenza potenziale [1] [2] .
MPI	1994	(Sì), (Sì), Sì	Multi CPU (Memoria Distribuita)	Implicita, Esplicita	Soluzioni scalabili; adottabile su diverse architetture [3] [4] .
PThread	1995	(Sì), (Sì), Sì	Multi-core CPU (Memoria Condivisa)	Esplicita, Implicita	Parallelismo esplicito; controllo totale (implicito, sebbene di basso livello) [2] .
OpenMP	1997	(Sì), (Sì), Sì	Multi-core CPU (Memoria Condivisa)	Esplicita, Implicita	Facile da imparare; scalabile; codice eseguibile sequenzialmente (implicanza) [2] .
CUDA	2007	(Sì), No, (Sì)	CPU + GPU(s)	Implicita(Esplicita), Implicita(Esplicita)	Accesso alla potenza di calcolo delle GPU [2] .
OpenCL	2008	(Sì), No, Sì	Architettura Eterogenea	Esplicita/Implicita, Esplicita/Implicita	Standard indipendente dall'hardware; nasconde dettagli architetturali [2] .
Apache Spark	2014	(Sì), No, (Sì)	Multi CPUs (Memoria Distribuita)	Implicita, Implicita	API per diversi linguaggi; non richiede parallelizzazione esplicita [5] [6] .

advanced approaches: mixing technologies and DSLs

- **OpenMP + CUDA**
  - this combination exploits both Multi-core CPUs and GPUs
    - CUDA is used to parallelize the code executed on the GPU
    - OpenMP is used to parallelize the code running on the CPU
- **MPI + OpenMP**
  - this mix supports two main scenarios
    - **first scenario**
      - MPI is used to express coarser parallelism (across Multi-CPUs)
      - OpenMP is used for finer parallelism (across Multi-cores)
    - **second scenario**
      - MPI is used primarily to implement communications
      - OpenMP is used to parallelize the computation
- **OpenCL + Verilog/VHDL**
  - hardware kernels (e.g., implemented on FPGAs) can serve as accelerators
    - OpenCL describes the parallelism among different processing elements
    - Verilog/VHDL is used to describe the underlying hardware kernel itself

domain specific languages (DSLs): Halide

- a sophisticated approach is the use of **Domain Specific Languages (DSLs)**, such as Halide
- Halide is characterized by the separations of the algorithm being implemented from its execution schedule
  - the **algorithm** defines what is computed (the required mathematical computation)
  - the **execution schedule** defines how the computation is performed efficiently
    - covering aspects such as loop nesting, parallelization, loop unrolling, and vector instructions
- this separation allows the programmer to easily experiment with scheduling to find the most efficient configuration without altering the core algorithm
  - Halide was initially developed for image processing but is now also employed in deep learning applications